

Architecture

Stack: Node.js + Express (TypeScript, MVC) | MongoDB + Mongoose | React SPA (customer) | EJS SSR (admin)

Repository layout

This is a monorepo — backend and frontend live side by side, with shared documentation at the root.

```
petcare-platform/
├── backend/      Express API + EJS admin panel
├── frontend/     React SPA
├── docs/         Problem statement, data model, architecture
├── .gitignore
└── README.md
```

Backend structure

```
backend/
├── src/          TypeScript source only
│   ├── controllers/  accept requests, return responses
│   │   ├── member.controller.ts
│   │   ├── pet.controller.ts
│   │   ├── product.controller.ts
│   │   ├── order.controller.ts
│   │   ├── chat.controller.ts
│   │   └── admin.controller.ts
│   ├── models/       business logic (service layer)
│   │   ├── Member.service.ts
│   │   ├── Pet.service.ts
│   │   ├── Product.service.ts
│   │   ├── Order.service.ts
│   │   ├── Chat.service.ts
│   │   └── AI.service.ts
│   ├── schema/       Mongoose schemas
│   │   ├── Member.model.ts
│   │   ├── Pet.model.ts
│   │   ├── Product.model.ts
│   │   ├── Order.model.ts
│   │   ├── OrderItem.model.ts
│   │   ├── ChatSession.model.ts
│   │   ├── ChatMessage.model.ts
│   │   ├── View.model.ts
│   │   └── Like.model.ts
│   ├── libs/
│   │   ├── types/    TypeScript interfaces (DTOs)
│   │   ├── enums/    enum definitions
│   │   ├── config.ts  constants, multer setup
│   │   ├── Errors.ts  error codes and messages
│   │   └── auth.ts    JWT sign and verify
│   ├── middlewares/
│   │   ├── auth.middleware.ts  JWT check for the SPA
│   │   └── admin.middleware.ts session check for the admin panel
│   ├── router.ts      SPA routes (JSON API)
│   ├── router-admin.ts admin routes (EJS)
│   ├── app.ts         Express configuration
│   └── server.ts      entry point
├── views/            EJS templates – not source code
│   ├── pages/
│   └── partials/
├── public/           css, client-side js, static images
├── uploads/          user-uploaded images
├── dist/             tsc output (git-ignored)
├── .env              secrets (git-ignored)
├── .env.example      template (committed)
├── package.json
└── tsconfig.json
```

Frontend structure



SPA endpoints (JSON API)

All responses are JSON. Protected routes require `Authorization: Bearer <token>`.

Member

Method	Endpoint	Auth	Purpose
POST	/member/signup	—	Register
POST	/member/login	—	Log in, returns JWT
GET	/member/detail	yes	Fetch own profile
POST	/member/update	yes	Update profile, including image

Signup accepts `memberNick`, `memberPhone`, `memberPassword`, and optionally `memberAddress`. Both signup and login return the member object plus an access token.

Pet

Method	Endpoint	Auth	Purpose
POST	/pet/create	yes	Register a pet
GET	/pet/all	yes	List own pets
GET	/pet/:id	yes	Single pet detail
POST	/pet/update	yes	Edit pet details
POST	/pet/delete	yes	Soft delete (status → DELETE)

Create accepts `petName`, `petType`, `petGender`, `petAge`, and optionally `petBreed`, `petWeight`, `petNotes`, `petImage`.

Product

Method	Endpoint	Auth	Purpose
GET	/product/all	—	Catalog, filtered and paginated
GET	/product/:id	—	Product detail, records a view

Query parameters: `page`, `limit`, `order` (`createdAt` / `productPrice` / `productViews`), `productPetType`, `productCategory`, `search`.

The `productPetType` filter is the core differentiating feature — the catalog narrows itself to the customer’s registered pet rather than showing everything.

Order

Method	Endpoint	Auth	Purpose
POST	/order/create	yes	Create an order from cart contents
GET	/order/all	yes	Order history, filterable by status
POST	/order/update	yes	Change order status (e.g. cancel)

Create accepts `petId`, an `items` array (each with `productId`, `itemQuantity`, `itemPrice`), and `orderDelivery`.

Chat (AI assistant)

Method	Endpoint	Auth	Purpose
POST	/chat/session/create	yes	Start a conversation about a pet
GET	/chat/session/all	yes	List conversations
GET	/chat/session/:id	yes	All messages in one conversation
POST	/chat/message	yes	Send a message, receive the reply

What happens on the server when a message arrives:

- 1. Resolve `memberId` and `petId` from the session
- 2. Load that pet’s profile from the database
- 3. Load the last few orders placed for that pet
- 4. Load prior messages in the conversation
- 5. Send all of it to the Claude API as context alongside the new message
- 6. Persist the reply to `chatMessages` and return it

Likes (optional, if time permits)

Method	Endpoint	Auth	Purpose
POST	/product/like	yes	Toggle like on a product

Admin endpoints (EJS, server-rendered)

These routes return **HTML pages**, not JSON. Authentication is session-based via `express-session` rather than JWT, since the browser loads these pages directly.

Method	Endpoint	Purpose
GET	/admin	Login page
GET	/admin/signup	Admin registration page
POST	/admin/login	Authenticate, open session
GET	/admin/logout	End session
GET	/admin/dashboard	Overview statistics
GET	/admin/product/all	Product list
POST	/admin/product/create	Create product, with image upload
POST	/admin/product/:id	Edit product or change its status
GET	/admin/member/all	Member list
POST	/admin/member/edit	Change member status (BLOCK / ACTIVE)
GET	/admin/order/all	All orders
POST	/admin/order/edit	Change order status
GET	/admin/pet/all	Registered pets, for statistics

Design decisions

The admin panel uses EJS, not React

The admin panel is internal-facing. It needs no SEO, no complex client-side state, and no real-time updates. Server-side rendering covers the requirement and builds considerably faster than a second SPA with its own auth flow, routing, and store would. This is a deliberate architectural choice, not a shortcut taken under time pressure.

It also means the project demonstrates both approaches — SPA where interactivity matters, SSR where it does not.

Two authentication strategies

The SPA sends a JWT in the `Authorization` header. The admin panel uses a server-side session, because the browser requests those pages directly and there is no client-side code to attach a header.

Static assets live outside `src/`

`tsc` compiles `.ts` files into `dist/` and copies nothing else. CSS, EJS templates, and images are assets, not source, so placing them inside `src/` would leave them missing after a build and produce 404s in production.

Paths therefore resolve one level up:

```
path.join(__dirname, "..", "public")
```

This works identically whether the process runs from `src/` under `ts-node` or from `dist/` after a build.

Each layer has one job

Layer	Responsibility	Does not
<code>controllers/</code>	Read <code>req</code> , write <code>res</code>	Query the database
<code>models/</code> (services)	Business logic, database access	Touch <code>req</code> or <code>res</code>
<code>schema/</code>	Describe document shape	Hold logic
<code>libs/types/</code>	TypeScript interfaces	Execute anything

The most common mistake here is writing Mongoose queries directly inside a controller. It is quicker in the moment and progressively harder to test or change as the project grows.

Build order

1. Repository skeleton, `.env`, MongoDB connection, Mongoose schemas
2. Member — signup, login, JWT
3. Pet — CRUD, once auth works
4. Product — catalog with filters, admin creation
5. Order — cart and order creation
6. Admin panel (EJS) — product and order management
7. React SPA — screens wired up one at a time
8. Chat (AI) — last
9. Testing, deployment, README

Each step builds on the one before it. The AI assistant comes last specifically because the context it depends on — pet profiles and order history — has to exist before there is anything meaningful to send.